

SUMMER 2026



LONGHORN HAPKIT

Lab guide #3

ROSS NEUMAN

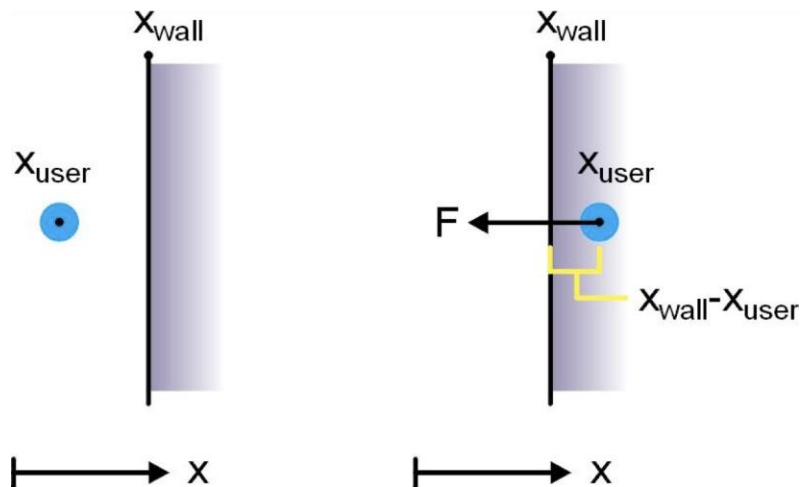
Assistant Professor of Practice, The University of Texas at Austin

Enhancing realism

- Once your velocity is dialed and damping feels good to you, you can use your new time-based signals to add realism to some previous environments
- We can also move on to dynamic simulations

Recall your 1 Degree of Freedom Wall

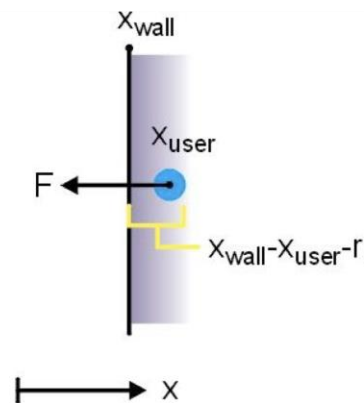
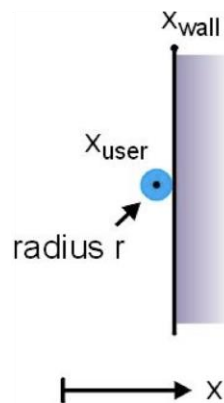
If $x_{user} > x_{wall}$, $F = k(x_{wall} - x_{user})$
 stiffness $k > 0$



Discussion

1. In what ways does this algorithm feel like a real wall?
2. How does it not feel like a wall?
3. How could you make it feel more like a wall?

If $(x_{user} + r) > x_{wall}$, $F = k(x_{wall} - x_{user} - r)$
 stiffness $k > 0$

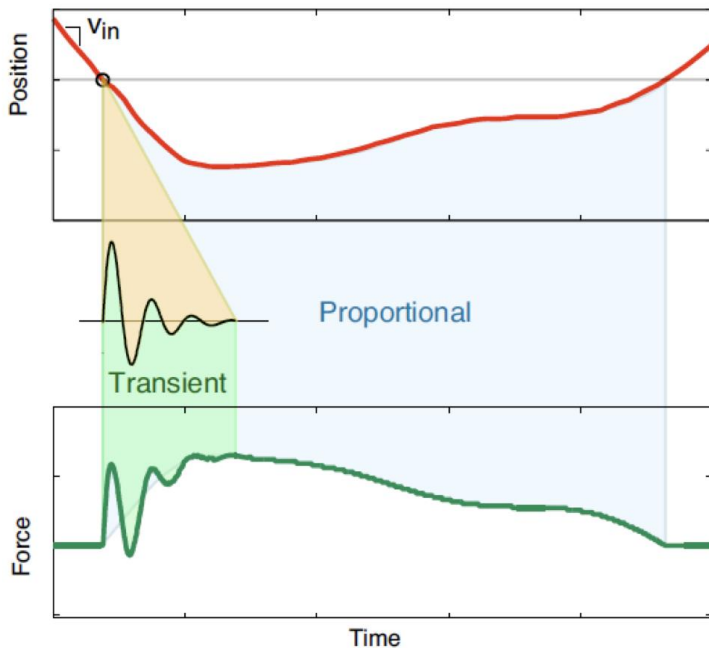
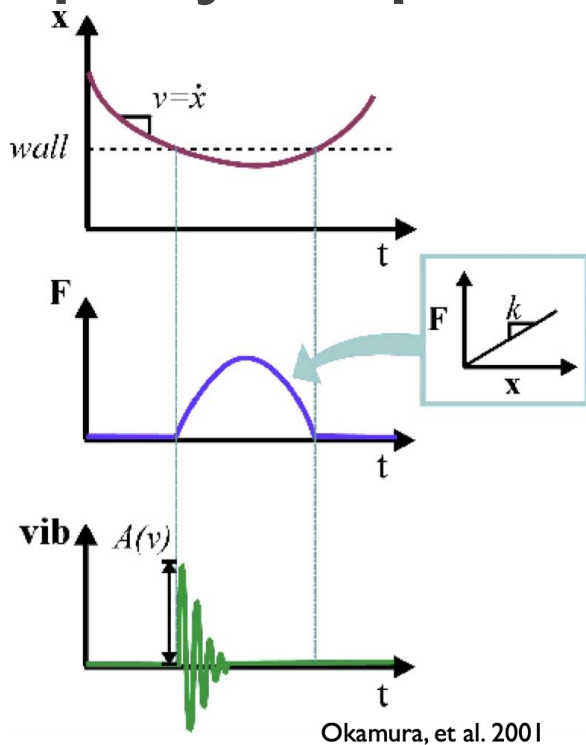


Damping for Virtual Walls

- a pure spring force for a wall may seem to “active”
- add a dissipative term, where b is the damping coefficient
- only damps when going into the wall
- this can also create vibrations upon wall impact

$$F = \begin{cases} k\Delta x + b\dot{x} & \text{for } \dot{x} > 0 \\ k\Delta x & \text{for } \dot{x} < 0 \end{cases}$$

Display impact vibrations!



Kuchenbecker, et al. 2006

Event-Based Haptics *CANVAS #10, Kuchenbecker*

- Add some additional event upon object collision to give impression of stiffer object.

- Fixed-Width Pulse

$$F_{pulse} = A|v_{in}| \text{ for } 0 < t \leq d.$$

- Decaying Sinusoid

$$F_{sine} = A|v_{in}|e^{\ln(0.01)t/d} \sin(2\pi ft) \text{ for } 0 < t \leq d.$$

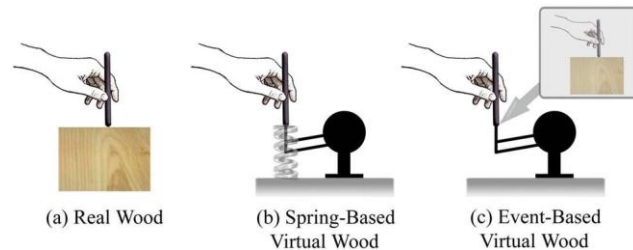


Figure 1. Real and virtual contact.

Decaying sinusoid code

- Declare these variables

```
// =====  
// Hard Surface  
// =====  
const float E_Value = 2.71828;  
  
const float Decay_Sine_Amplitude_Scale = 5.0;  
const float Decay_Sine_Amplitude_Max = 20.0;  
  
float Decay_Sine_Amplitude = 0;  
  
const float Decay_Sine_Frequency = 150.0;  
const float Decay_Sine_Exponent = -20.0;  
  
int Decay_In_Wall_Flag = 0;  
float Decay_Sine_Time = 0;
```


Decaying sinusoid code

- Then for your calculations

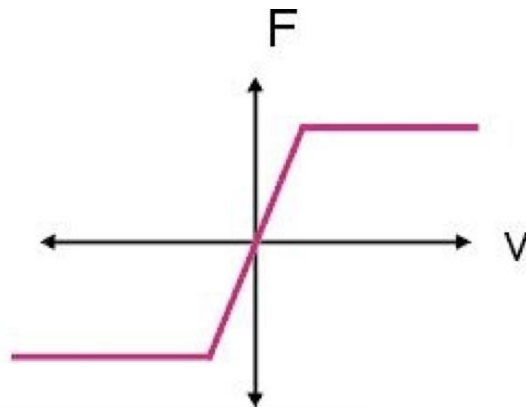
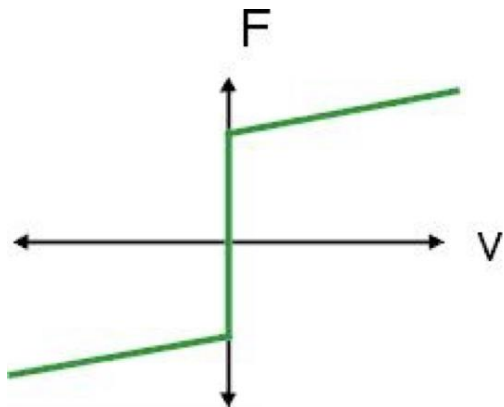
```
if (xh > x_wall) {  
    if (Decay_In_Wall_Flag == 0) {  
        Decay_Sine_Time = 0;  
  
        Decay_Sine_Amplitude = Decay_Sine_Amplitude_Scale * fabs(vh);  
  
        if (Decay_Sine_Amplitude > Decay_Sine_Amplitude_Max)  
            Decay_Sine_Amplitude = Decay_Sine_Amplitude_Max;  
  
        Decay_In_Wall_Flag = 1;  
    }  
  
    motorCommand = -exp(Decay_Sine_Exponent * Decay_Sine_Time) * Decay_Sine_Amplitude * sin(2.0 * PI * Decay_Sine_Frequency * Decay_Sine_Time);  
  
    motorCommand += -k_wall * (xh - x_wall);  
  
    Decay_Sine_Time += dt;  
} else {  
    Decay_In_Wall_Flag = 0;  
    Decay_Sine_Time = 0;  
    Decay_Sine_Amplitude = 0;  
    motorCommand = 0;  
}
```

Decaying sinusoid code

- Mess around with parameter values
 - Change frequency, change amplitude, etc.
- It's pretty subtle at present but you can make it very noticeable
- Try to make the “realest” wall you can

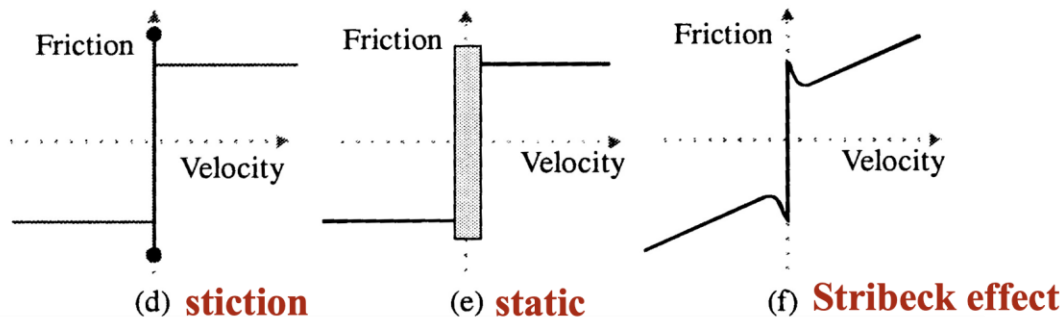
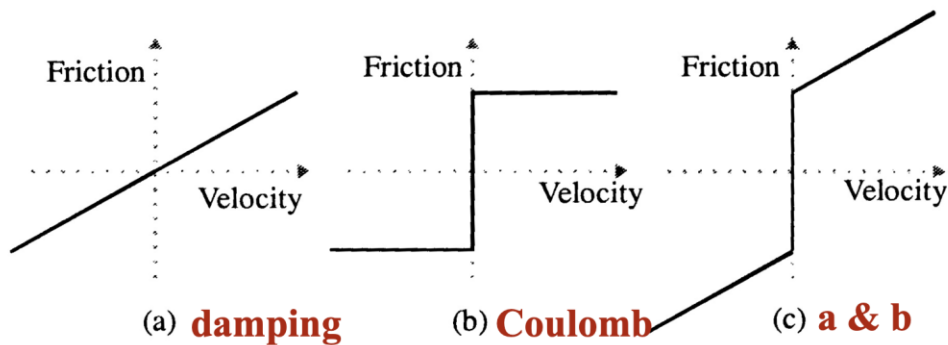
Frictional Damping

- surfaces can feel unnaturally slippery
- friction would help, but there are different ways to implement it
- Introduce a “sticky” feel when velocity is low



Different friction Displays

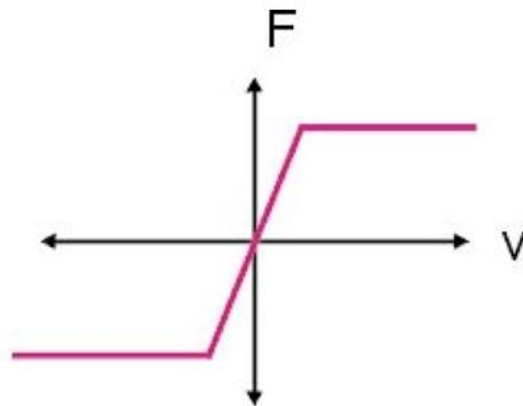
- Can be difficult to render due to non-linearity



Logic for coulomb friction

$$F = \begin{cases} -F_c, & v_h > v_{th} \\ -kv_h, & |v_h| \leq v_{th} \\ +F_c, & v_h < -v_{th} \end{cases}$$

- F_c = friction force magnitude
- v_{th} = small deadband threshold
- k = damping near zero velocity



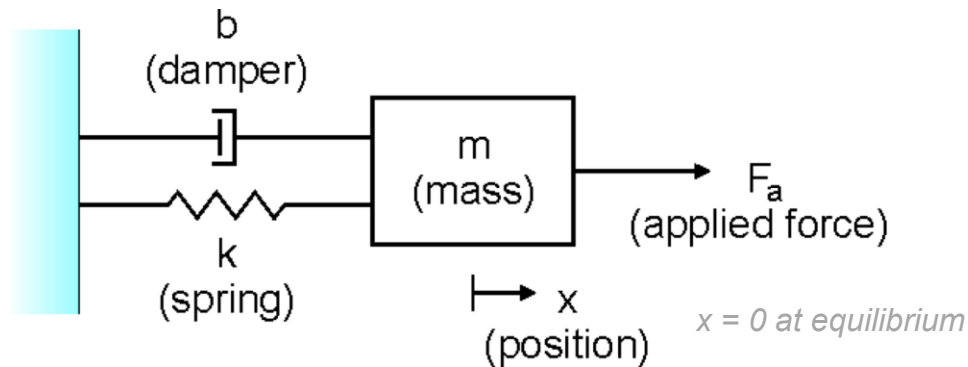
Dynamic Simulation of “Rigid” Bodies

- assumptions
 - you have an impedance-type force-feedback display
 - you are using a linear stiffness model of the surface
- basic approach:
 1. save the state of the moving object
 2. sum the forces on the object
 3. calculate the new state

Object State

- the “state” of an object is used to describe it’s current condition
- made up of variables that will change with time, such as

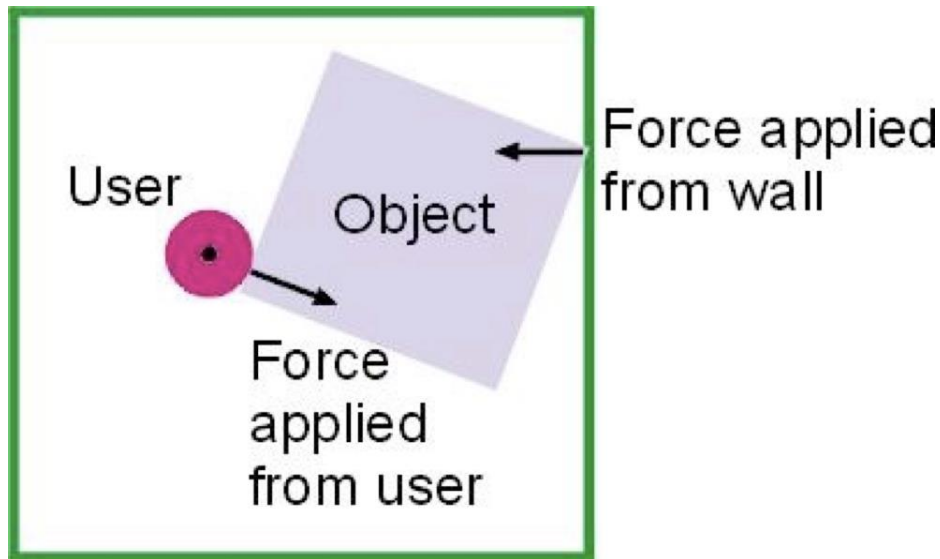
- position
- velocity
- Acceleration



- rarely: other parameters such as mass, shape, etc. that might be changed by dynamic interaction

Calculate the Forces on an Object

- for forces from the user's hand pushing, this is equal and opposite to the force fed back to the user
- for forces from other objects in the VE, use the same idea: force is proportional to penetration



The new state

- now you have a new position, velocity, and acceleration of the object
- use new position for collision detection, penetration and force calculation
- store velocity and acceleration for the next integration
- do the loop again

Pseudocode

- `store last_ObjAcc, last_ObjVel`
- `ObjForce = Force_from_user + Force_from_other_objects`
- `Force_from_user = k*penetration_distance_user`
- `Force_from_other_objects =
k*penetration_distances_objects`
- **can add other effects (e.g. damping) as well**
- `ObjAcc = ObjForce/mass` ($F=ma \rightarrow a = F/m$)
- **integrate using (for example) the trapeziodal rule:**
`ObjVel += 1/2 (last_ObjAcc + ObjAcc)/sim_freq`
`ObjPos += 1/2 (last_ObjVel + ObjVel)/sim_freq`

Real code

```
// =====  
// Mass-Spring-Damper  
// =====  
  
float m_msd = 3.0;  
float b_msd = 1.0;  
float k_msd = 20.0;  
float k_user = 100.0;  
  
float x_msd = 0.5;  
float v_msd = 0;  
float a_msd = 0;  
  
float xi_msd = 0.5;
```

Real code

```
{  
    v_msd += a_msd * dt;  
    x_msd += v_msd * dt;  
  
    if (xh > x_msd) {  
        a_msd = (k_user * (xh - x_msd) + k_msd * (xi_msd - x_msd) - b_msd * v_msd) / m_msd;  
        motorCommand = k_user * (x_msd - xh);  
    } else {  
        motorCommand = 0;  
        a_msd = (k_msd * (xi_msd - x_msd) - b_msd * v_msd) / m_msd;  
    }  
  
    break;  
}
```